
DEVELOPMENT DOCUMENTATION

2. Architecture

Modules, data model, lifecycle and technology

Dynamic Auctioneers Marketing Platform

Built by Keegan Haumann, Cognexa

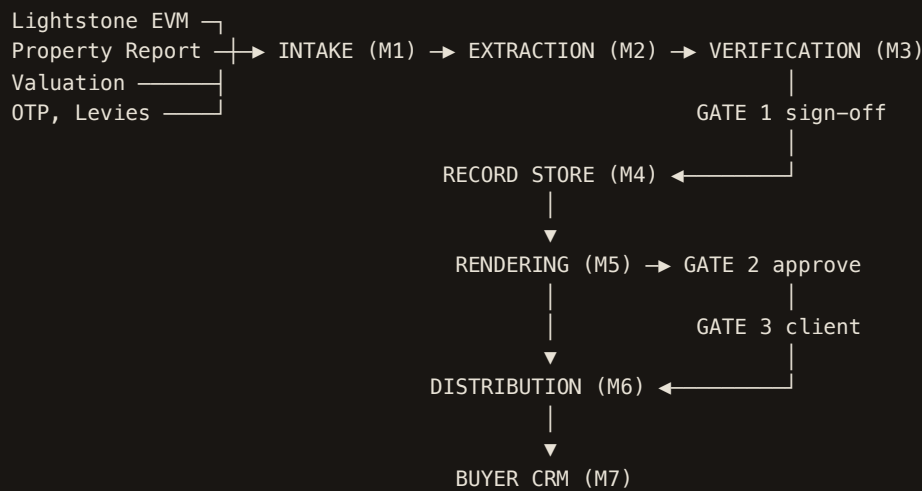
Compiled 20 August 2026 · code state **9c9e0df** on main

2.1 Shape of the system

Two halves that share one SQLite database:

- **engine/** — the pipeline. Pure Python, importable, CLI-driven, no web framework. Modules M1–M7.
- **webapp/** — the platform (M8). FastAPI + Jinja2 + HTMX. The face the team uses, and the only surface Dynamic ever touches.

The engine does not import the webapp. The webapp calls the engine. Long work (extraction, verification, rendering, posting) is pushed onto a jobs table and drained by one background worker, so no HTTP request ever waits on a model call.



Everything above sits behind the PLATFORM UI (M8).

2.2 The modules

MODULE	PURPOSE	CODE
M1 Intake	Classify uploaded documents by content, resolve the DP number, hold a job until the set is complete	<code>engine/intake.py</code>
M2 Extraction	Read the PDFs into a validated record	<code>engine/extract.py</code> , <code>engine/schema.py</code> , <code>engine/photos.py</code> , <code>engine/otp.py</code> , <code>engine/levies.py</code> , <code>engine/pdftext.py</code>
M3 Verification	Cross-check the sources, research the market, write the memo, hold gate 1	<code>engine/verify.py</code>
M4 Record store	Persist records, enforce the lifecycle, keep an audit trail	<code>engine/store.py</code>
M5 Rendering	Turn a record into the nine artifacts	<code>engine/render/</code>
M6 Distribution	Route to channels, post through GHL, build manual packs	<code>engine/distribute/</code>
M7 Buyer CRM	Tag enquiries by DP, match buyers to new listings	<code>engine/crm.py</code>
M8 Platform UI	The web application	<code>webapp/</code>
—	OneDrive/Graph watcher (optional, superseded by D20)	<code>engine/watch/graph.py</code>

2.3 The data model

The record

One `PropertyRecord` per DP number (`engine/schema.py`, 634 lines of Pydantic). `DP3060/record.json` is the worked reference. Top-level groups:

GROUP	SOURCE OF TRUTH	NOTES
identity	Lightstone (deeds)	Erf or scheme/unit, address, GPS, title deed, municipality
physical	Precedence, see below	Beds, baths, garages, extent, features, flatlet, portions, conflicts
valuation	Lightstone EVM, comparables, municipal	Plus valuation.professional — the valuer's market and forced-sale figures
financials_internal	Both	Owner, bond, arrears. Never rendered
sale_process	Property Report and OTP	Terms, method, viewing, auction logistics
marketing	System and human edits	Headline, price display, photos, template set, channel routing
compliance	System	PII redaction confirmations
verification	M3	Status, flags, sign-off

Source precedence on physical facts (D35)

Three sources can describe the same wall. The trust order is **valuation > property report > lightstone**, applied in code by `resolve_physical_conflicts`. Every disagreement is recorded as a structured `PhysicalConflict` carrying each source's value, and raises one blocking `PHYSICAL_CONFLICT` flag rather than being silently resolved. Gate 1 shows a source picker defaulting to the precedence winner, which a human can override.

Precedence covers **physical facts only**. Lightstone still owns deeds, legal and market data.

The two privacy layers

`PropertyRecord.public_view()` is the only thing a renderer or the copy model ever receives. It physically removes:

- **The POPIA internal layer** — owner name and ID, occupant contact, bond, arrears.
- **The sale-strategy layer** (`_strip_internal_strategy`) — municipal, professional, forced-sale and comparable valuations. This was written after the copy model was caught printing a municipal valuation onto a live advert (D54, D57).

A typed asking or offers figure is allowed everywhere; a *valuation* figure is allowed nowhere in buyer-facing output.

Human edits (`human_overrides`, D17) are applied **last** inside `public_view()`, so one edit reaches every artifact and survives re-renders.

2.4 The lifecycle

Enforced in `engine/store.py`; illegal moves raise `IllegalTransition` and are never silently ignored. Every transition is appended to a `state_events` table.



Notes on the real machine:

- `extracted` may go straight to `verified` when nothing is flagged, or to `flags_raised` when something is.
- `photos` is an explicit step inserted between gate 1 and the advert draft (D47, D52): photographs are mandatory before an advert is drafted, so no render is wasted. `verified → drafted` remains legal for programmatic paths.
- `updated` is the change fast-path. It can return to `live`, `approved` or `client_approved` depending on how far back the change has to go. Change requests after gate 3 are internal only — the client is not re-consulted (D13).
- `archived` is terminal.

2.5 Rendering

Rendering goes through a swappable backend contract (`engine/render/base.py`) so no renderer is welded into the engine.

BACKEND	STATE
<code>html</code>	The default and the one in use. Renders the brand-token Jinja templates locally. No credentials, always available
<code>canva</code>	A config-gated scaffold. Autofill against Canva brand templates. Proven working on the team's Canva Teams plan — Enterprise turned out not to be required (D16, D40)
<code>mixed</code>	Per-format routing between the two (D18)

Backend resolution is argument → `ENGINE_RENDERER` environment variable → default `html`. An unconfigured Canva backend reports its missing variables instead of crashing, and the whole backend can be removed by deleting one file, one registry line and one test file — a property the test suite proves.

Every backend receives `public_view` only, Canva included, because anything sent to a cloud service is client-facing by definition.

Rasterisation to PNG and PDF is headless Chromium via Playwright (`engine/render/rasterize.py`). The pack ships as PDFs, each printed at its own canvas (D67).

2.6 Distribution

The channel matrix is pure deterministic code with no external calls (`engine/distribute/routing.py`):

RULE	CHANNELS
Every property	Property24, own website, Facebook, email list
Valued at ≥ R10,000,000	Add JamesEdition
Industrial or commercial	Add commercial portals (specific portals still TBD)
Always excluded, by policy	Private Property

Value is read sensibly: an explicit numeric `marketing.price_display` first, then the EVM estimate, then municipal valuation, then comparables — so a textual display like "Offers invited" falls through rather than breaking the routing.

Posting runs through the GoHighLevel Social Planner (`engine/distribute/ghl.py`) to Facebook, Instagram and LinkedIn. Static images only; video is out of scope (D24). TikTok and broadcast messaging were removed (D19, D26). Photos and the advert PNG are hosted through the GHL Media Library and attached as CDN URLs (D30).

`GHL_POST_STATUS=draft` in the environment is a **hard guard rail**: it overrides any per-post choice made in the UI, so a misconfigured box cannot publish. The UI otherwise offers save-as-draft, schedule, or post now (D28).

Channels without an API get a **ready-to-post pack** (`engine/distribute/packs.py`) and the system tracks posted or not-posted per channel per version.

2.7 Technology

CONCERN	CHOICE	WHY
Language	Python 3.12	PyMuPDF, Pillow and the PDF tooling were already there
AI	Anthropic API, <code>claude-opus-4-8</code> , adaptive thinking	Extraction accuracy on messy PDFs is the whole game
Extraction shape	Sectioned structured output, one call per section, non-strict tool use	A strict grammar rejected even a flat section as too complex (D21, D23)
Web research	Server-side <code>web_search_20260209</code> tool	No scraper infrastructure to maintain
PDF reading	Native PDF blocks to the model; PyMuPDF word positions for the OTP and levy readers	See D77 below
Photos	PyMuPDF at source quality	Proven in phase 0
Store	SQLite	Single-operator scale, zero operations
Web	FastAPI + Jinja2 + HTMX	One Python stack, no JavaScript build chain
Auth	Session cookies + bcrypt; roles marketing / approver / admin	About five users; SSO is not worth it
Background work	SQLite jobs table + one worker loop	No Redis, no Celery
Rasterising	Playwright Chromium	Renders the grid-based advert faithfully
Hosting	One VPS, nginx TLS in front of uvicorn on <code>127.0.0.1:8000</code>	Zero operations, fits the budget

D77 is worth knowing about, because it is the kind of fault that hides: `engine/otp.py` and `engine/levies.py` originally shelled out to `pdftotext`, which does not exist on the production server. Every OTP and every levy statement would have read as "could not be read" *on the box only* — degrading politely, silently dead, on a machine nobody would check because the tests pass locally. Both now rebuild table rows from PyMuPDF word positions (`engine/pdftext.py`), which is already a hard dependency. One code path, same result everywhere.

2.8 Background jobs

`webapp/jobs.py`. Four kinds — `extract`, `verify`, `render`, `post` — claimed one at a time by a single `Worker` thread started at application startup and stopped at shutdown. Jobs are rows in the shared SQLite database, so a restart does not lose queued work. A failure parks the record with the raw output attached rather than letting anything proceed silently.

2.9 Repository layout

```
marketing-socialmedia-dynamicauctioneers/
├── SPEC.md                Module specs, acceptance criteria, decision log D1-D77
├── CLAUDE.md              Working brief and status ledger
├── README.md              Prototype-era overview
├── requirements.txt        Pinned runtime dependencies
├── .env / .env.example    Secrets (never committed) and the template
├── engine.db              SQLite database (untracked)
├── engine/                The pipeline
│   ├── cli.py             All CLI commands
│   ├── intake.py          M1 document classification, DP resolution
│   ├── extract.py         M2 the model calls
│   ├── schema.py          M2 the PropertyRecord model and public_view
│   ├── photos.py          M2 photo extraction
│   ├── otp.py / levies.py M2 sale terms and monthly levy readers
│   ├── pdftext.py         M2 PyMuPDF word-position row rebuilder
│   ├── aicache.py         content-addressed cache for the paid calls
│   ├── verify.py          M3 cross-checks, research, memo
│   ├── store.py           M4 records, state machine, audit trail
│   ├── crm.py             M7 enquiries and buyer matching
│   ├── render/            M5 backends, templates, rasterising
│   │   ├── base.py        the backend contract and FORMATS
│   │   ├── service.py     orchestration, price changes, edits, photos
│   │   ├── html_backend.py the default backend
│   │   ├── canva_backend.py the removable scaffold
│   │   ├── copy.py        channel-aware copy generation
│   │   ├── ad_templates.py the four advert designs
│   │   ├── rasterize.py   Chromium to PNG and PDF
│   │   └── templates/     the Jinja artifact templates
│   ├── distribute/        M6 routing, GHSL client, manual packs
│   └── watch/graph.py     optional OneDrive watcher (placeholder)
├── webapp/                M8 the platform
│   ├── main.py            application factory, middleware, worker lifecycle
│   ├── auth.py            bcrypt sessions, roles, admin seeding
│   ├── models.py          platform tables, settings, shared secret
│   ├── jobs.py            the job queue and worker
│   ├── tokens.py          signed single-use approve-by-email tokens
│   ├── ratelimit.py       sign-in throttling (D44)
│   ├── routes/            board, intake, gates, artifacts, post, settings, email_approve
│   ├── templates/         Jinja screens and partials
│   └── static/            app.css, app.js
├── docs/                  DESIGN-SYSTEM, INFO-PACK-PLAYBOOK, SERVER-ACCESS,
│                           deploy/, fixlist/, development/ (this folder)
├── designs/incoming/      client reference designs
├── scripts/               deploy.sh, Canva authorisation helpers
├── tests/                 22 test modules, 370 passing
└── DP3060/               the golden property: record, memo, photos, artifacts
```

Data lives outside git. `engine.db` and the `DP<dp>/` property folders are untracked, so a deployment `git pull` never touches them.